Machine Learning

> “ Computers are like humans – they do everything except think. ”



Often Attributed to John von Neumann

Machine Learning

Unsupervised Learning:



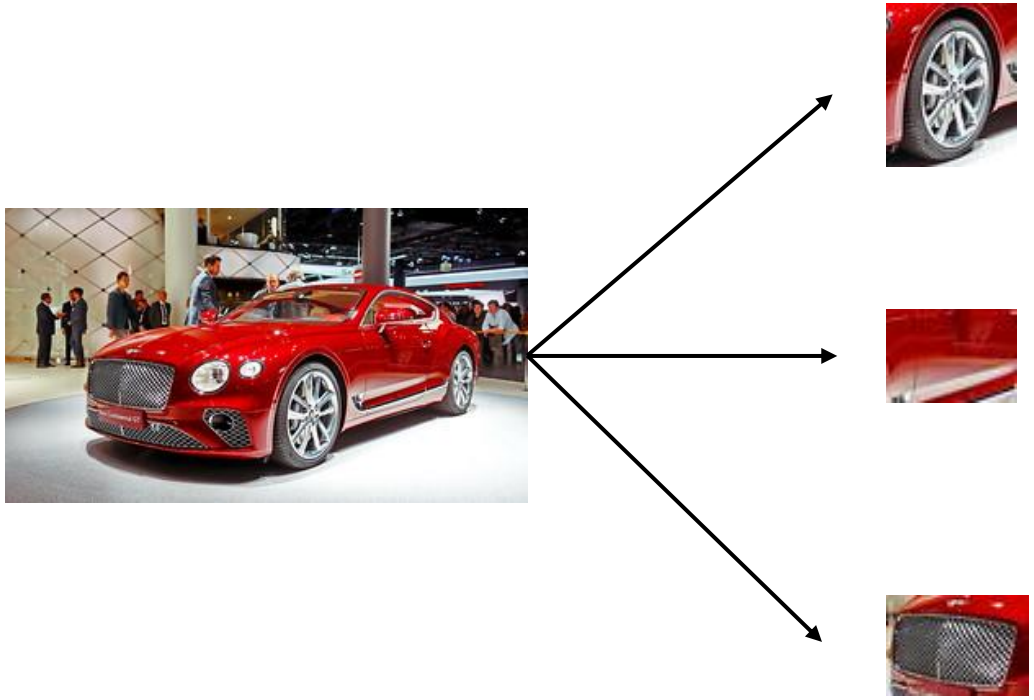






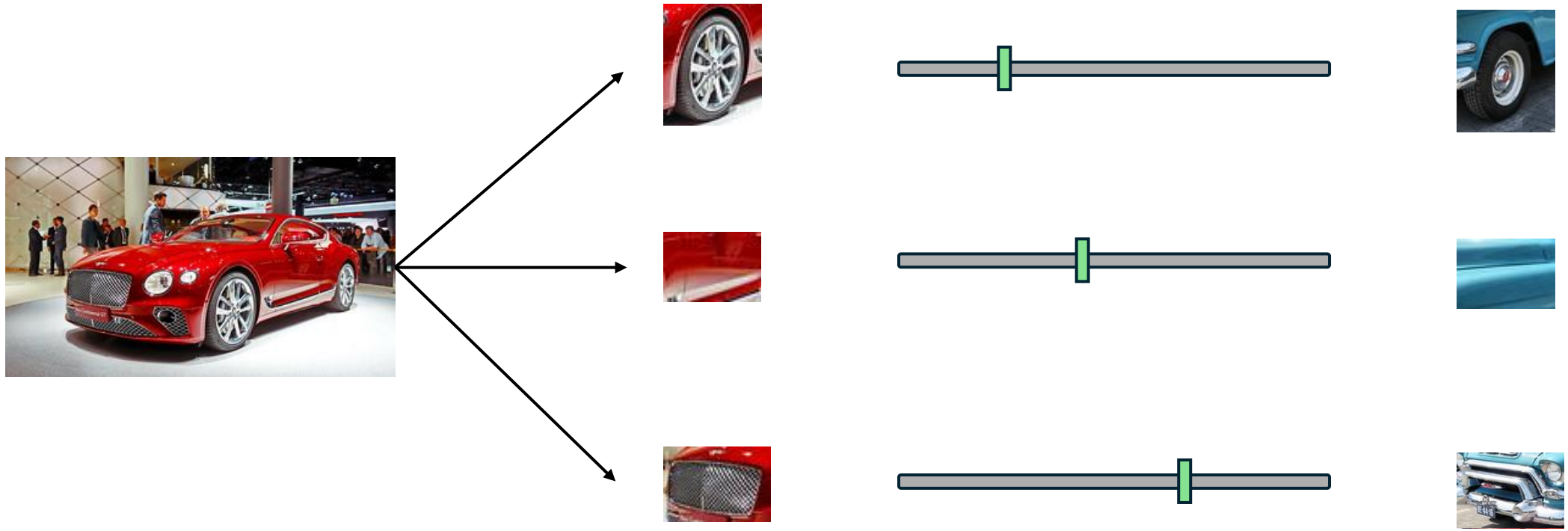

Machine Learning

Unsupervised Learning: Finding compressed representation

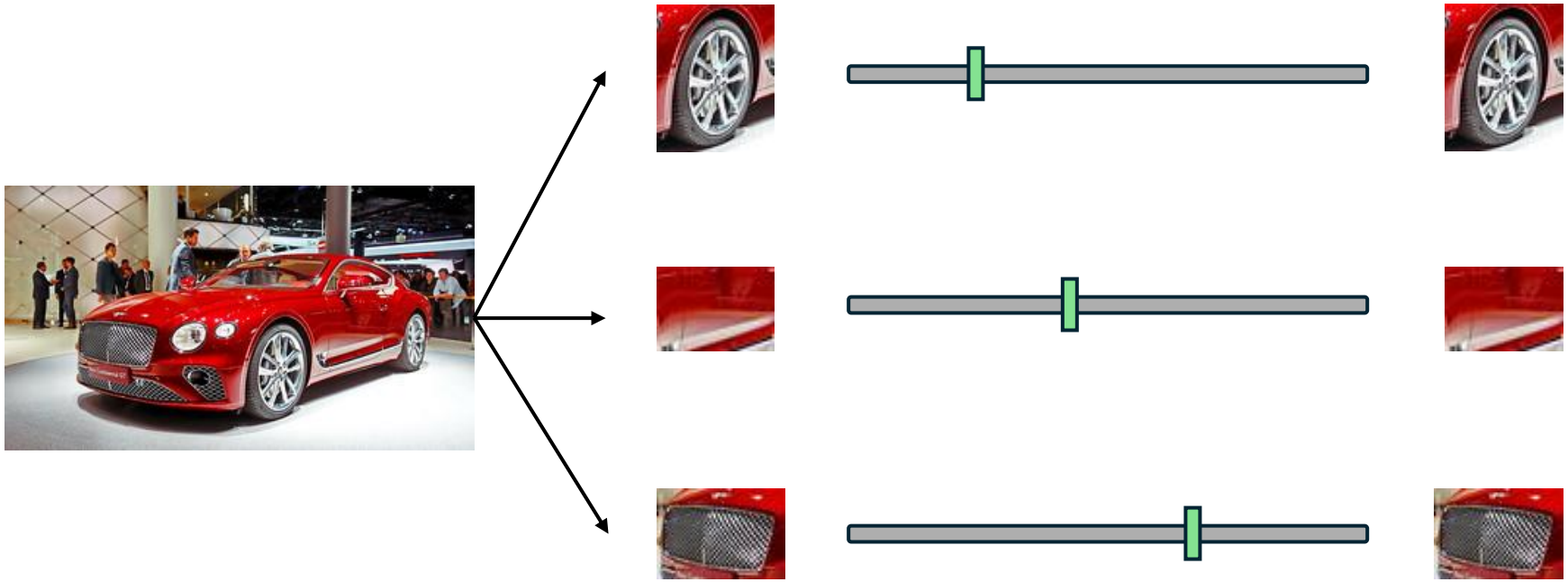


Machine Learning

Unsupervised Learning: Finding compressed representation



Autoencoders



Autoencoders



Encoder





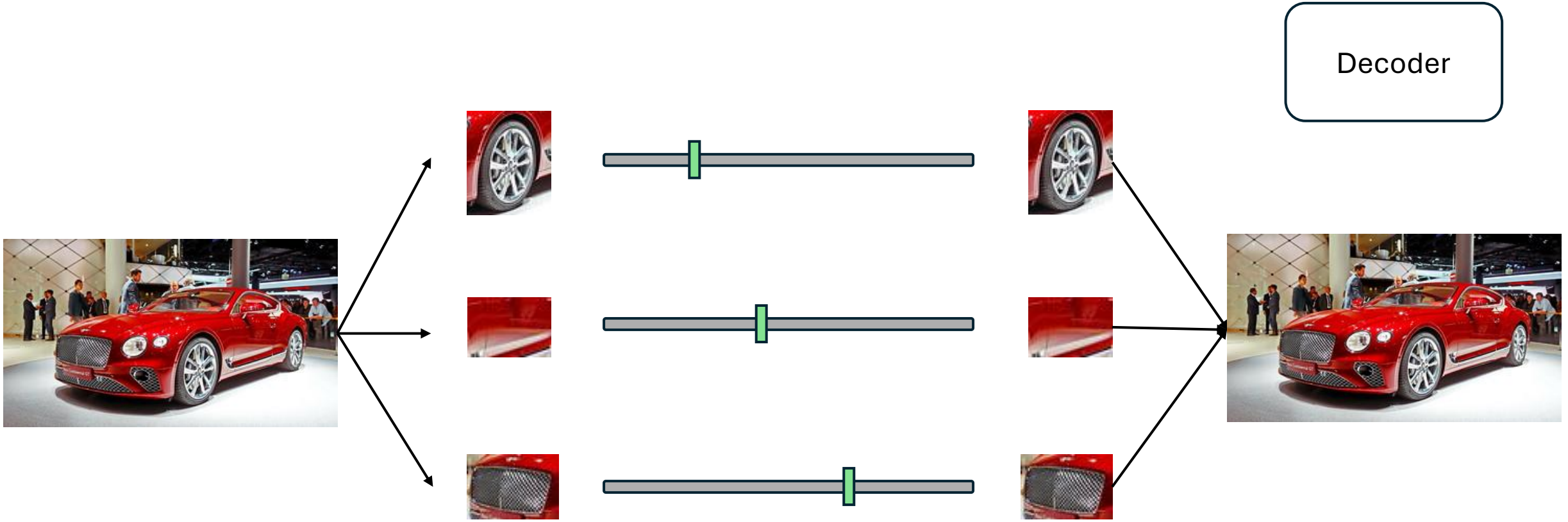


Two problems:

1- How do we decide what to extract?

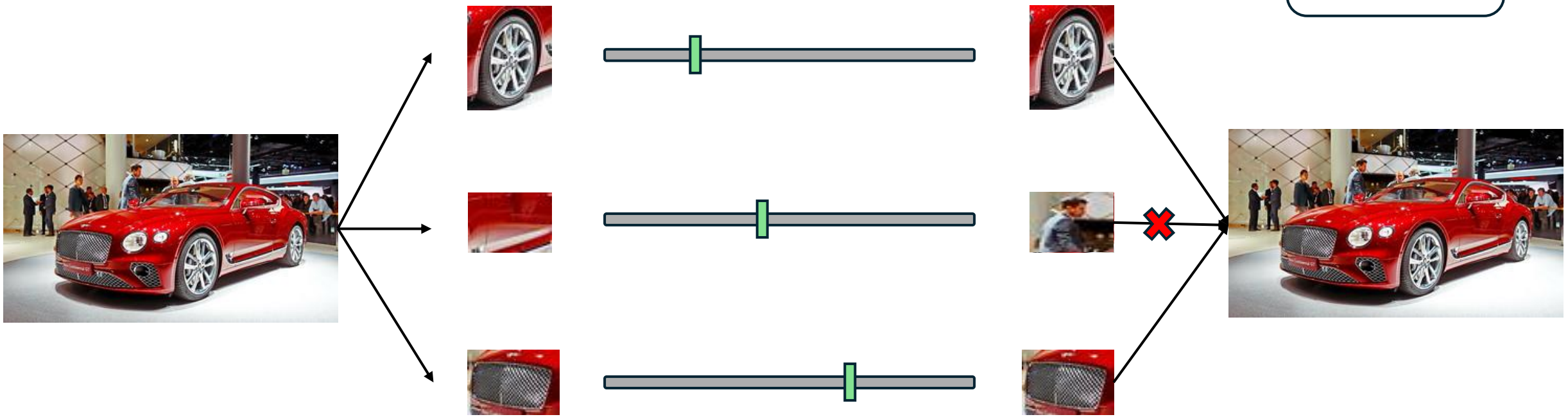
2- How to go from “new wheel” to “new car”?

Autoencoders



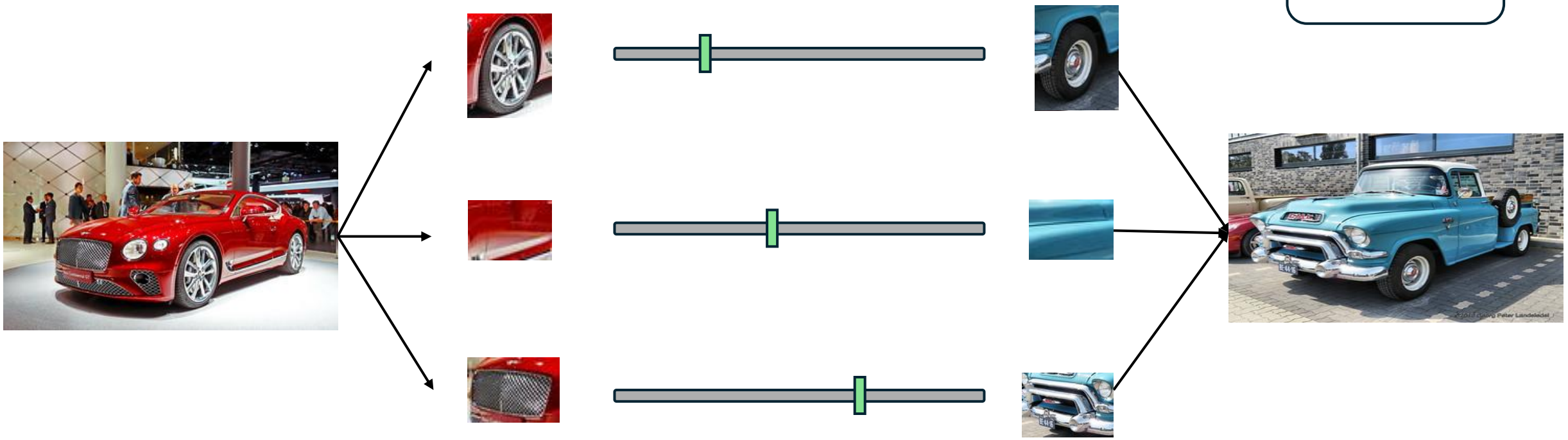
Autoencoders

Problem 1: How do we decide what to extract?

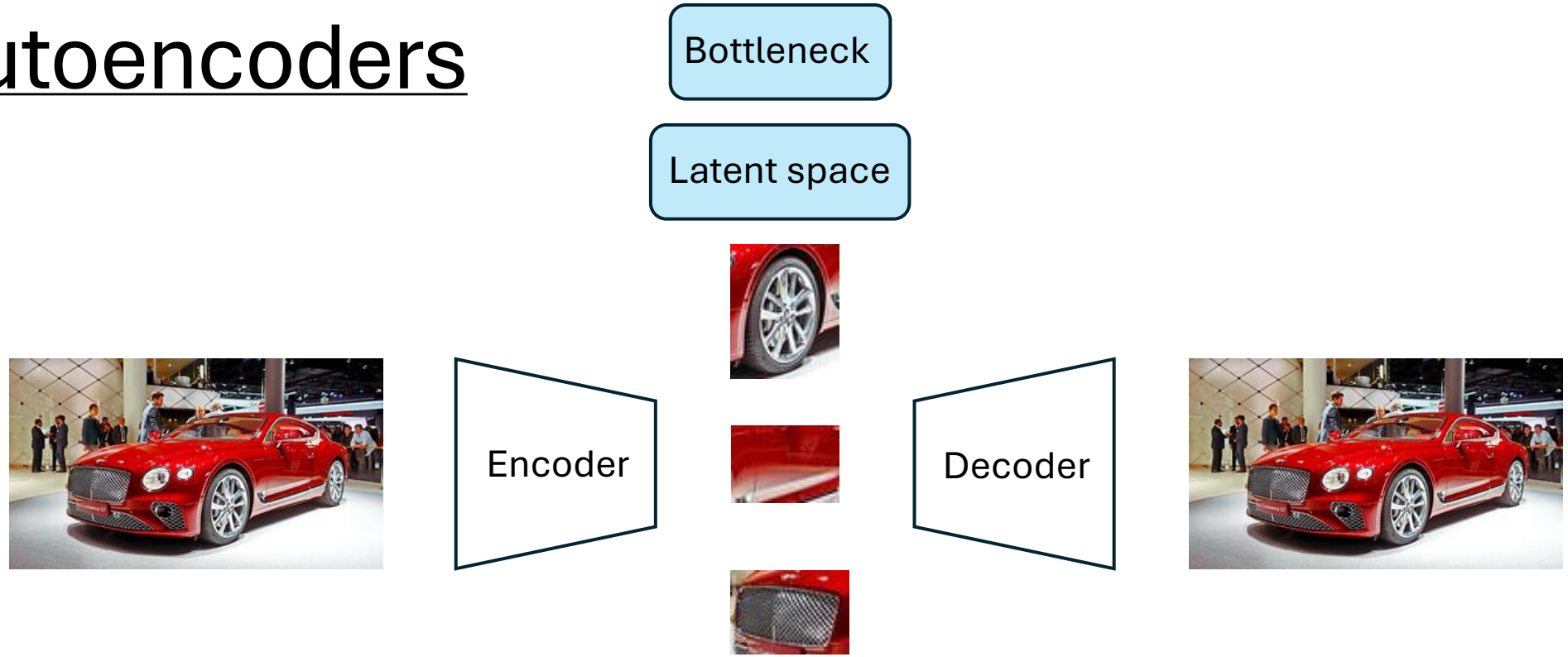


Autoencoders

Problem 2: How to go from “new wheel” to “new car”?

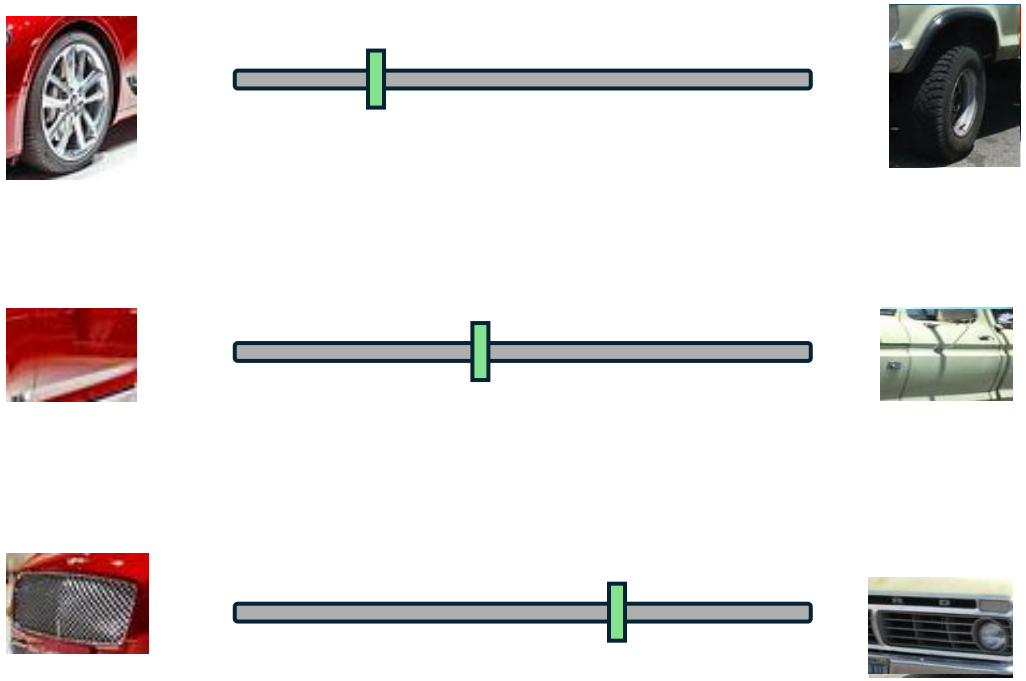


Autoencoders

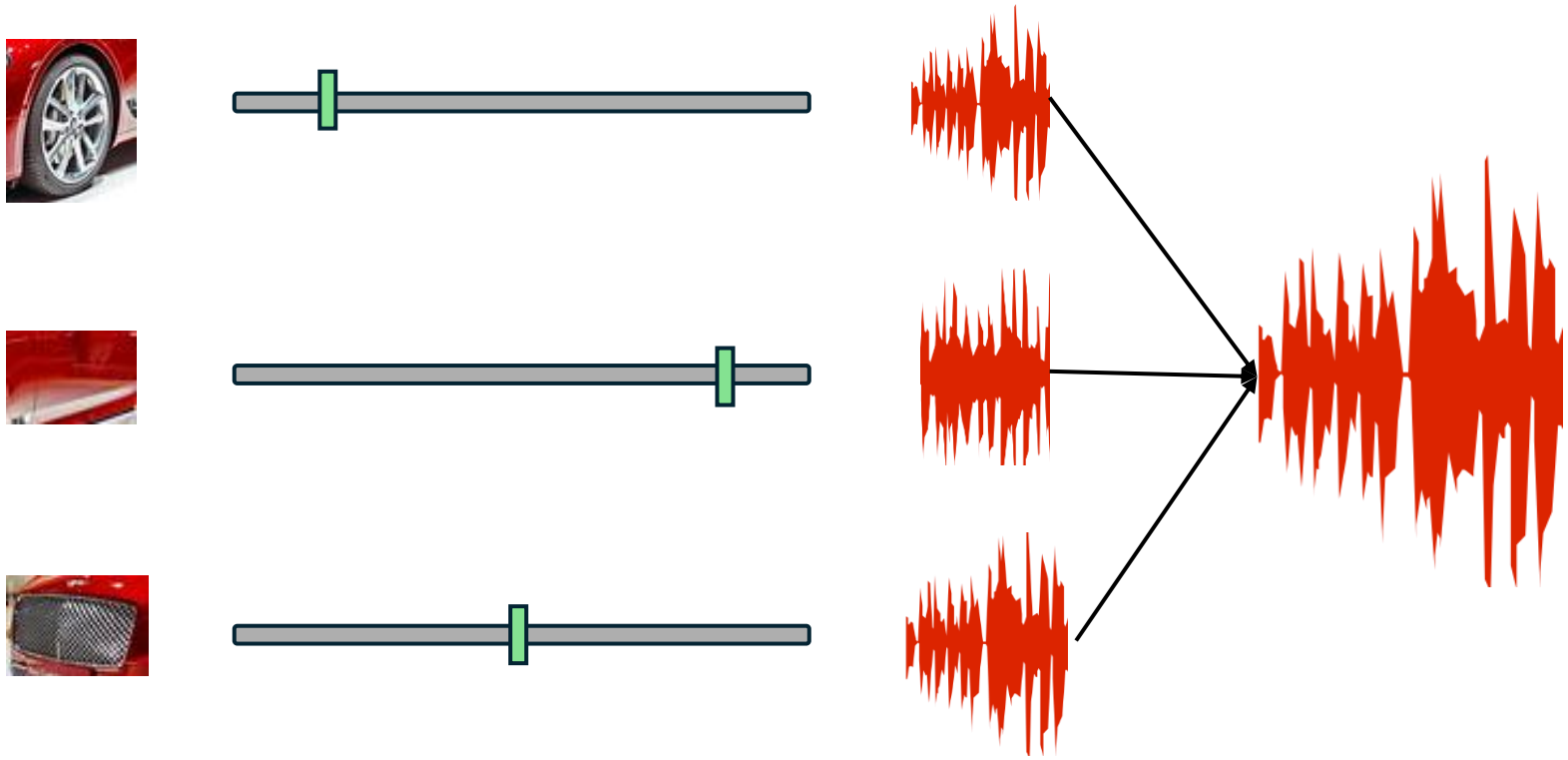


Loss: Input = Output

Autoencoders' Limitations



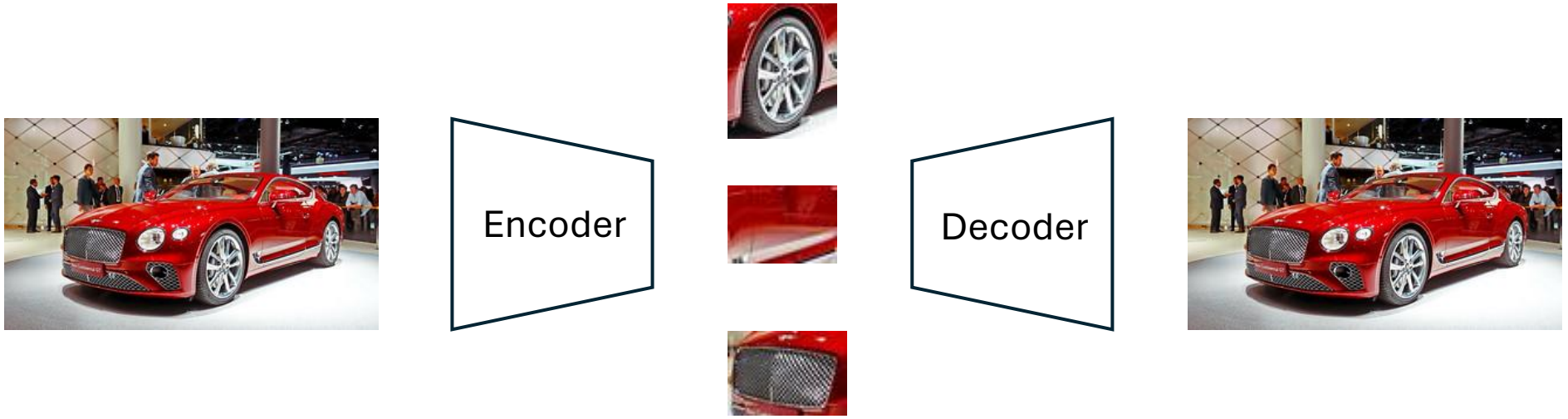
Autoencoders' Limitations



Autoencoders' Limitations

Proposed enhancements in the literature:

a- L1 Regularization



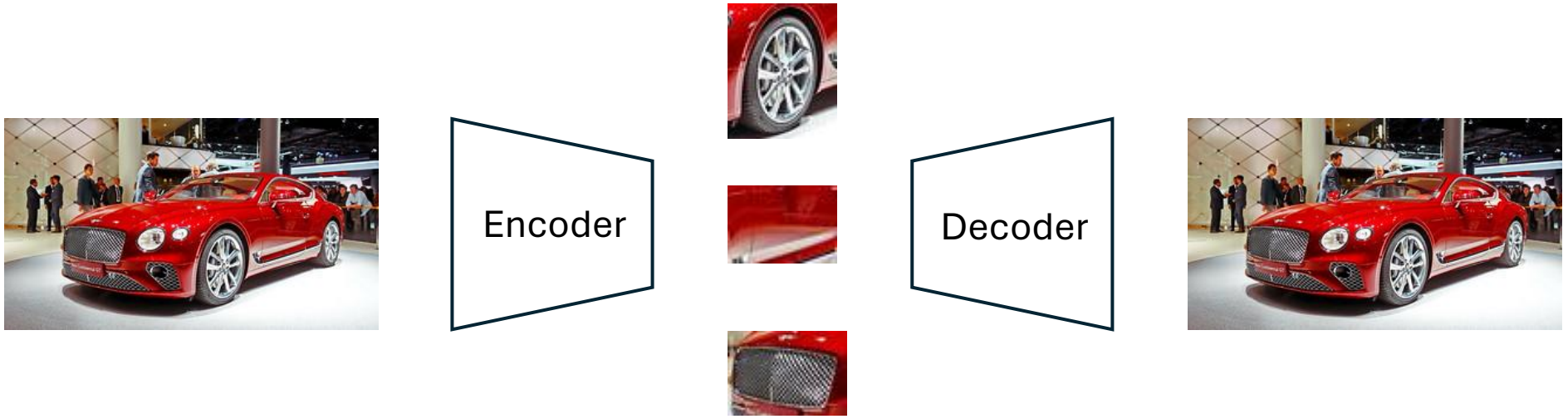
Loss: Input = Output

Loss: $||W_{encoder}||_1$, and $||W_{decoder}||_1 \downarrow$

Autoencoders' Limitations

Proposed enhancements in the literature:

b- L2 Regularization



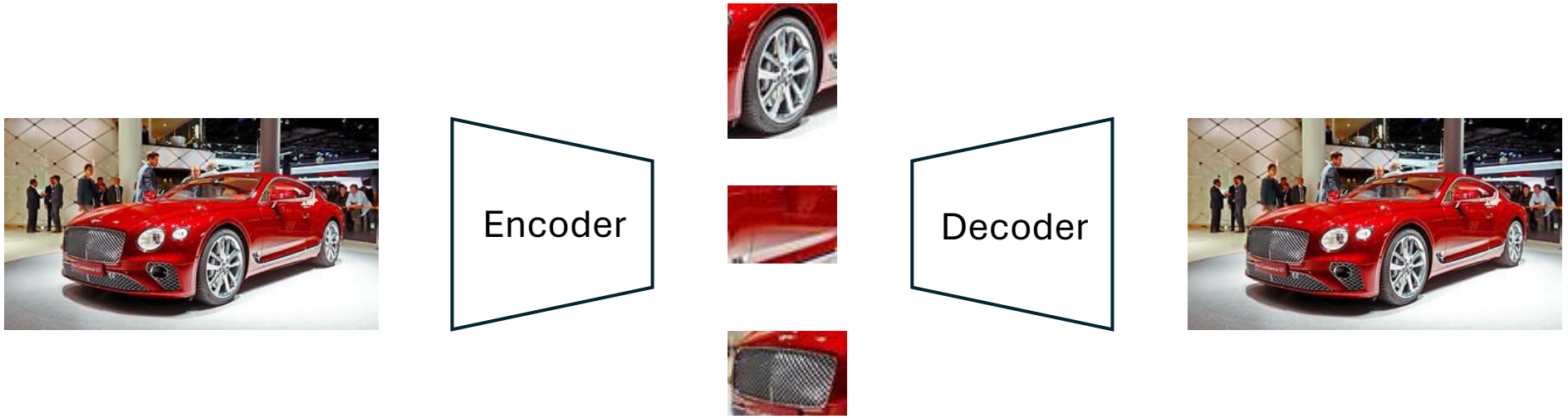
Loss: Input = Output

Loss: $||W_{encoder}||_2$, and $||W_{decoder}||_2 \downarrow$

Autoencoders' Limitations

Proposed enhancements in the literature:

c- Sparse Autoencoders



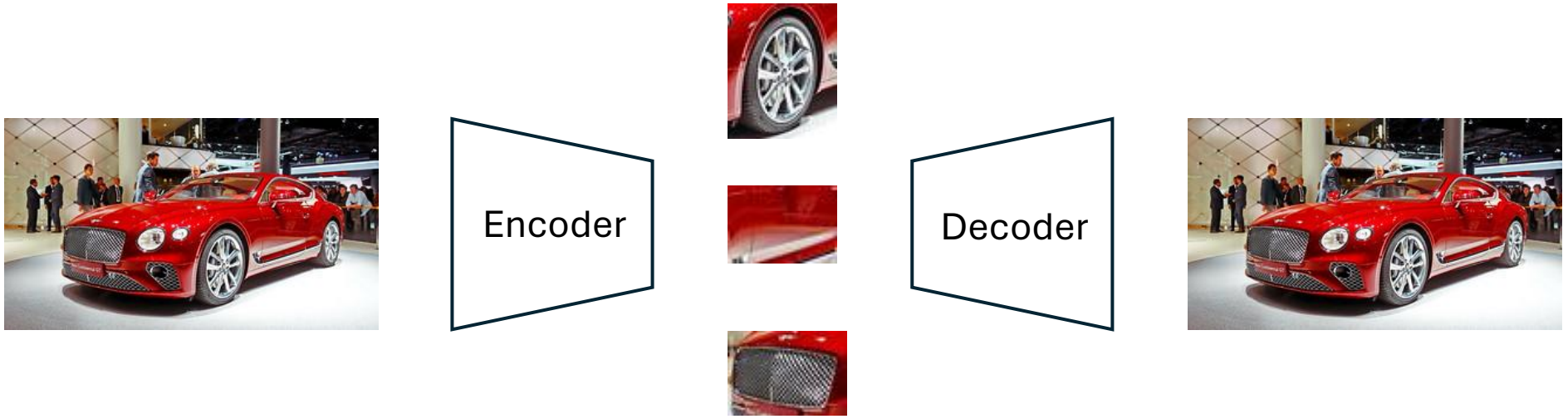
Loss: Input = Output

Loss: Sparsity of the latent space (close to zero).

Autoencoders' Limitations

Proposed enhancements in the literature:

d- Contractive Autoencoders



Loss: Input = Output

Loss: Latent Jacobian ↓

Autoencoders' Limitations:

Proposed enhancements in the literature: Opitmizing more things!!

Next: An autoencoder that can be regularized while only optimizing input = output

Rank Reduction Autoencoders (RRAEs)

Truncated Singular Value Decomposition (SVD):

For any matrix $Y \in \mathbb{R}^{m \times N}$, and a chosen integer $k^* \leq m$, we can write:

$$Y \approx Y^{k^*} = U\Sigma V^T = U\alpha,$$

with, $U \in \mathbb{R}^{m \times k^*}$, $\alpha \in \mathbb{R}^{k^* \times N}$

Three important notes:

1- Reduction from m to k^*.

2- $\alpha = \Sigma V^T$ has a very specific structure (sorted and regularized).

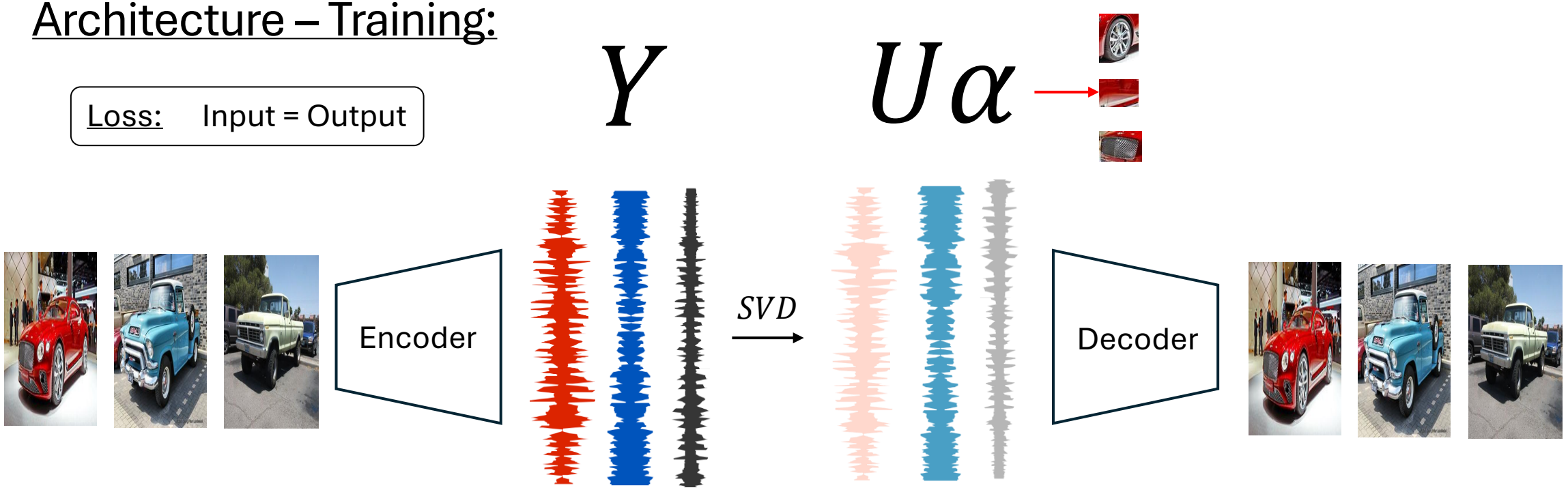
3- If we use Y^{k^*}, we never see the dimension k^*.

Rank Reduction Autoencoders (RRAEs)

Architecture – Training:

Loss: Input = Output

Y $U\alpha$



1- Bottleneck exists.

2- Regularized by the SVD.

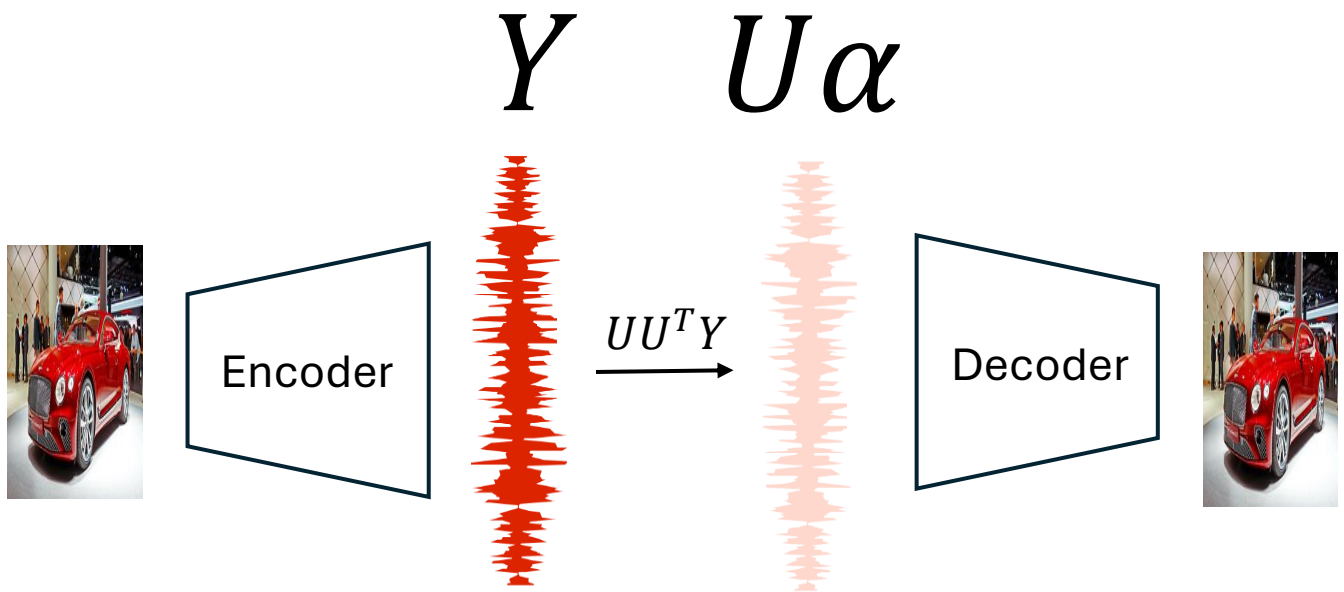
3- Only one loss.

4- No dependence on k^*.

5- Sorted Bottleneck.

Rank Reduction Autoencoders (RRAEs)

Architecture – Evaluation (U saved after training):



Rank Reduction Autoencoders (RRAEs)

Paper contribution:

1- The first paper to use the SVD and its gradient in Machine Learning training.

```
@stable_SVD.defjvp
def _svd_jvp_rule(primals, tangents):
    (A,) = primals
    (dA,) = tangents
    U, s, Vt = jnp.linalg.svd(A, full_matrices=False)

    s = (s / s[0] * 100 >= 1e-9) * s      -->  Added line

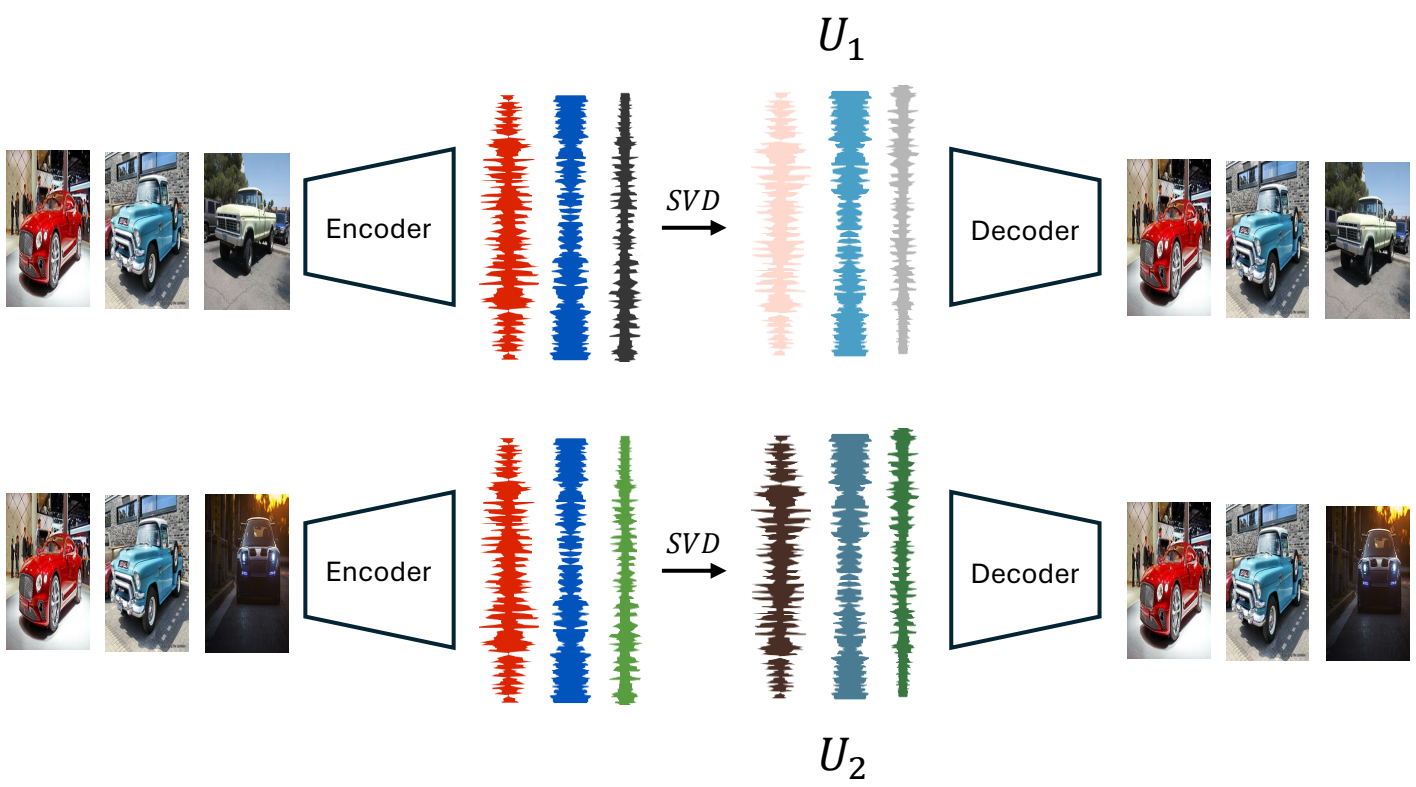
    Ut, V = _H(U), _H(Vt)
    s_dim = s[..., None, :]
    dS = Ut @ dA @ V
    ds = _extract_diagonal(dS.real)

    s_diffs = (s_dim + _T(s_dim)) * (s_dim - _T(s_dim))
    s_diffs_zeros = (jnp.abs(s_diffs) <= 1e-9).astype(s_diffs.dtype) --> Changed (was == 0)
```

Rank Reduction Autoencoders (RRAEs)

Paper contribution:

2- Proof of convergence of basis U when training with batches.



$$U_1 \approx QU_2$$

Rank Reduction Autoencoders (RRAEs)

Paper contribution:

3- Adaptive Algorithm that allows choosing k^* automatically.

4- No dependence on k^*.

5- Sorted Bottleneck.

Algorithm 1 Adaptive Rank Reduction Autoencoder Algorithm

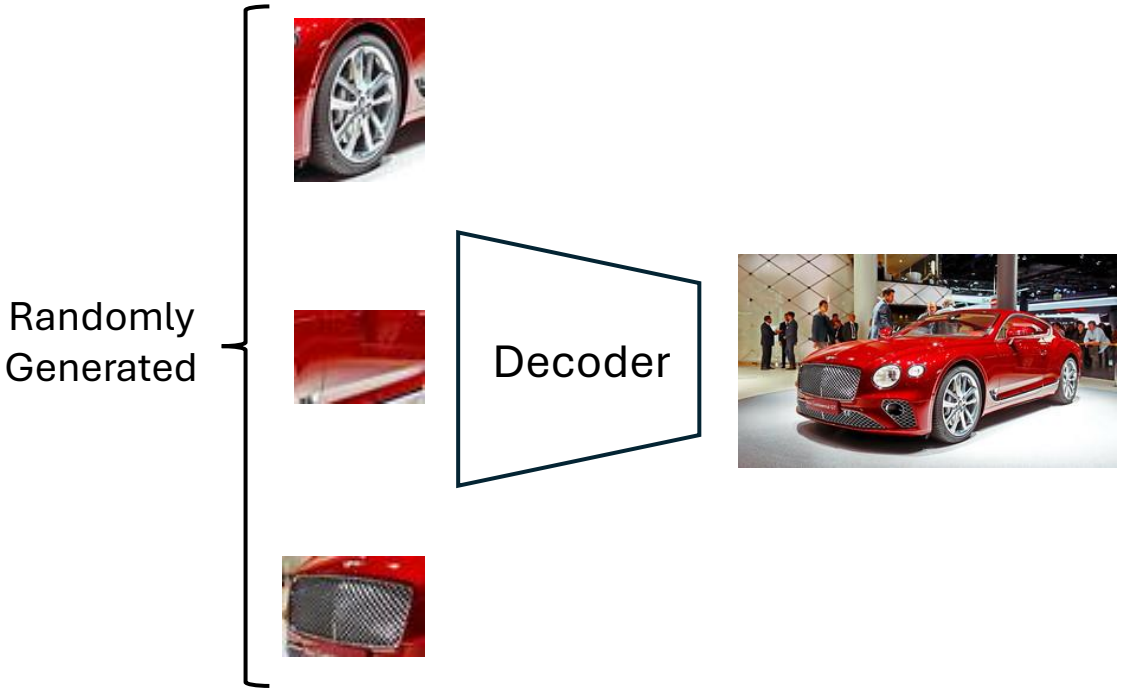
Input: Unlabeled Data $X \in \mathbb{R}^{D \times N}$, stagnation criteria.
Initialize $k^* = \min(D, L)$ (or $\min(B, L)$ if bacthed).
Train RRAEs as described in Section 4 until convergence.
Store the final epoch's loss as the *optimal_loss*.
while True **do**
  $k^* = k^* - 1$.
  Continue training the RRAE with the new k^* until one of the following,
  if *optimal_loss* reached **then**
    Save model at k^*.
  else if stagnated **then**
    $k^* = k^* + 1$.
    Load model saved at k^*.
    Break.
if batches are used **then**
  Find the final training basis U_f as described in Subsection 4.2.
  Fine tune the decoder using U_f.
else
  Find the final training basis U_f as in Figure 1.
Output: Trained RRAE with the optimal bottleneck size, and $U_f \in \mathbb{R}^{L \times k^*}$.

Adaptive RRAEs (aRRAEs)

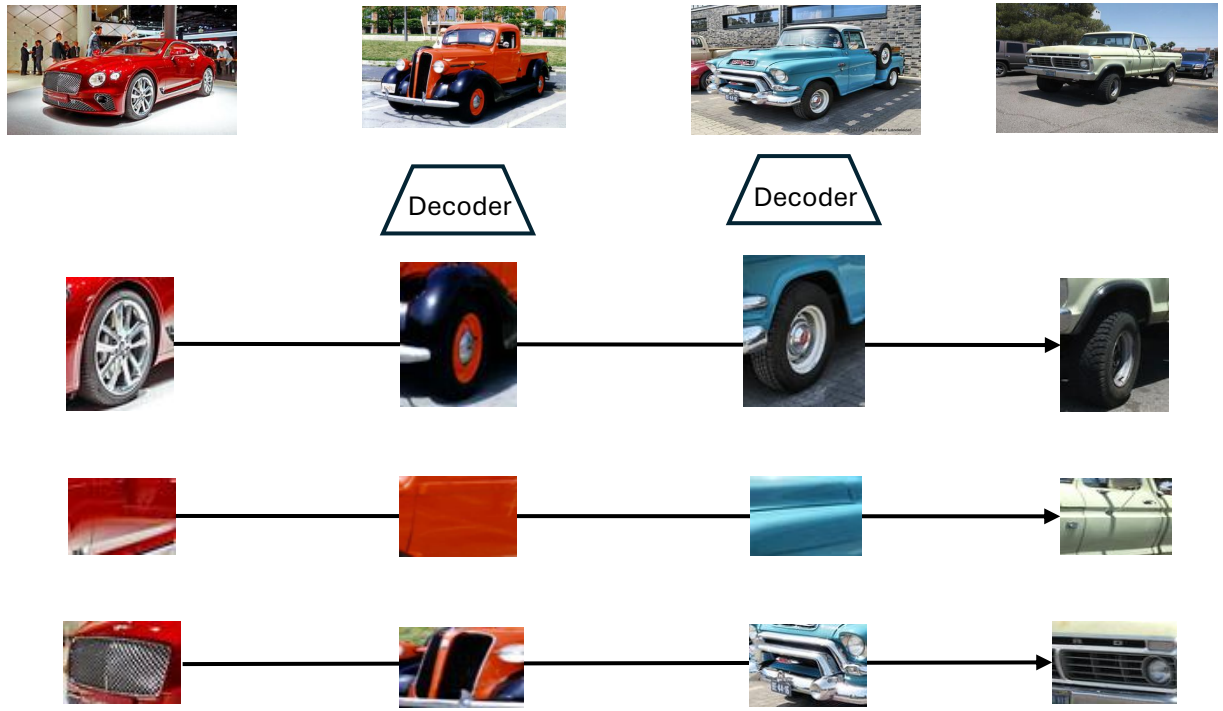
Rank Reduction Autoencoders (RRAEs)

Paper contribution – Results :

Random Generation



Interpolation



Rank Reduction Autoencoders (RRAEs)

Paper contribution – Results :

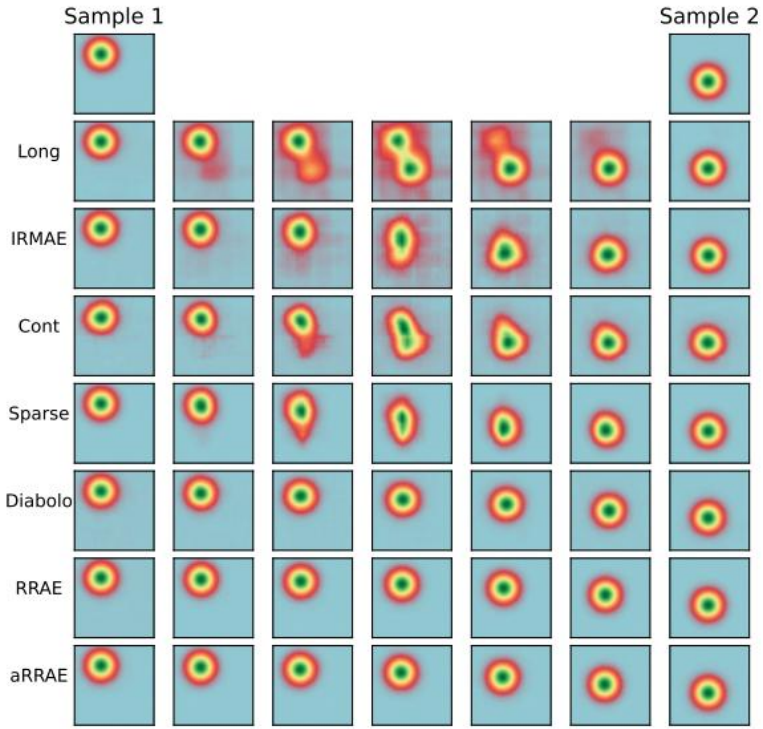


Figure 3: Interpolation for different architectures on the 2D gaussian problem.

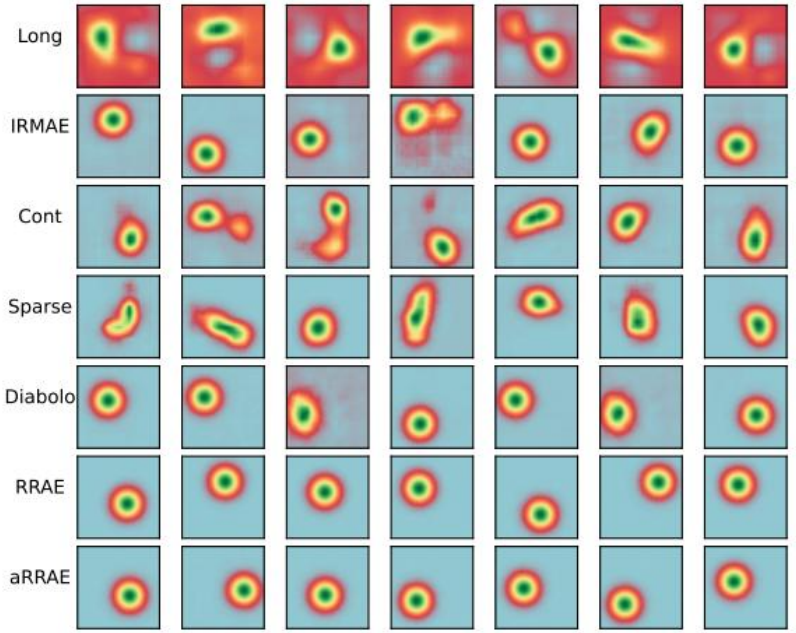


Figure 4: Random sampling for different architectures on the 2D gaussian problem.

Rank Reduction Autoencoders (RRAEs)

Paper contribution – Results :



Figure 7: Example of randomly sampled CelebA images using all Autoencoders presented in the paper.

Rank Reduction Autoencoders (RRAEs)

Paper contribution – Results :

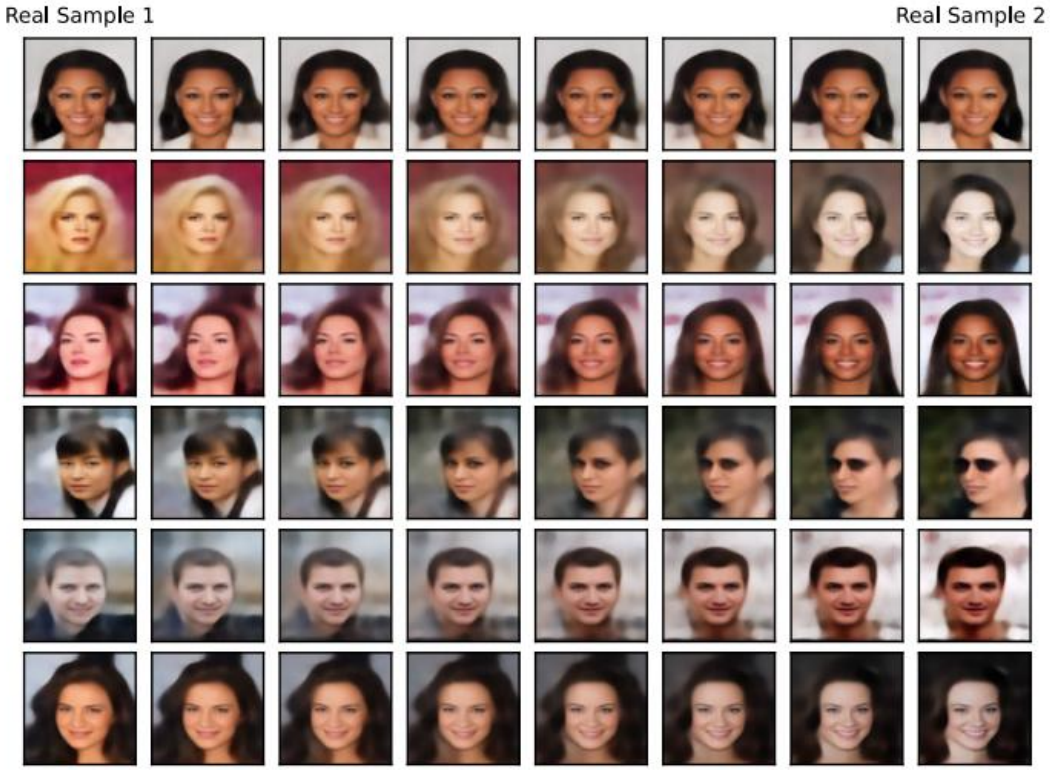


Figure 6: Example of interpolation between different samples using aRRAEs. Other models are not presented since their reconstruction is blurry. The picture to the right in the first row is the real image flipped along the vertical axis.

Rank Reduction Autoencoders (RRAEs)

Paper contribution – Results :

Model	Train Error	Test error	Interpolation	Random
Long	1.95	2.07	26.46 ± 15.2	63.29 ± 12.39
IRMAE	3.66	3.78	20.02 ± 18.68	39.17 ± 16.87
Cont	10.76	12.04	27.51 ± 17.4	57.81 ± 16.13
Sparse	**0.92**	**1.21**	17.29 ± 19.96	50.94 ± 21.15
Diablo	2.66	2.45	1.93 ± 0.34	4.28 ± 4.79
RRAE (fixed)	**0.98**	**1.14**	1.03 ± 0.39	1.78 ± 1.2
aRRAE	0.99	1.27	**0.982 ± 0.08**	**1.26 ± 0.43**

Table 1: Relative error (in %) for different architectures on the 2D gaussian problem. The error represents the difference between the original bump and the reconstruction (columns 1 and 2), and the difference between the fitted bump and the generated one (columns 3 and 4). Mean and standard deviations for generated samples are reported over 250 samples. A horizontal line in the middle splits AEs that don't enforce a bottleneck (top) and those who do (bottom).

	MNIST		Fashion-MNIST		CelebA	
Model	Interp.	Random	Interp.	Random	Interp.	Random
Long	10.16	86.5	35.21	67.13	16.53	17.23
IRMAE	8.09	42.58	23.2	43.62	15.79	15.47
Cont	30.5	90.5	27.51	55.32	16.74	17.36
Sparse	**6.1**	57.67	61.32	87.2	16.91	16.57
Diablo	8.92	44.5	23.6	48.29	15.76	15.75
RRAE (fixed)	7.45	41.5	23.41	40.6	14.89	15.07
aRRAE	6.6	**41.42**	**22.31**	**40.21**	**5.74**	**5.91**

Table 2: Table documenting FID values for 20000 images generated by Interpolation/Random sampling. Standard deviations were not given as they were all of a small order.